

DEVOPS TECHNICAL ASSIGNMENT REPORT

Production-like Node.js Deployment using AWS EC2, Docker and GitHub Actions

Abstract

This report documents the complete implementation of a production-like Node.js web application deployment on Amazon Web Services (AWS). The goal of the assignment was to go through the typical steps a DevOps engineer follows when shipping an application: setting up cloud infrastructure, containerising the application, automating deployment through a CI/CD pipeline, monitoring the running service, storing backups, and verifying that the system can handle load.

The application is a simple Node.js web server hosted on an AWS EC2 t3.micro instance (Ubuntu 26.04 LTS). Docker 29.1.3 is used to package the application and Nginx 1.28.3 acts as the reverse proxy. GitHub Actions automates the build and deployment whenever code is pushed to the main branch. Amazon CloudWatch provides CPU monitoring and alerting, while Amazon S3 holds a compressed backup of the project directory. Load testing was performed using k6 with 20 virtual users over 30 seconds, producing 600 total requests with an average response time of 2.02 ms and a 0% error rate.

The report covers architecture design, technology choices, step-by-step implementation details, challenges encountered during setup, measured results, and suggested improvements for a production-grade deployment.

Table of Contents

1. Introduction
2. Project Objectives
3. Technology Stack
4. Project Workflow
5. Step-by-Step Implementation
 - 5.1 AWS EC2 Setup
 - 5.2 Docker Configuration
 - 6.3 Node.js Application
 - 5.4 GitHub Repository & Actions CI/CD
 - 5.5 Amazon S3 Backup
 - 5.6 CloudWatch Monitoring
 - 5.7 Load Testing with k6
6. Security Configuration
7. Challenges Faced
8. Results
9. Future Improvements
10. Conclusion
11. References

1. Introduction

DevOps is a set of practices that combines software development and IT operations with the aim of shortening the development cycle and delivering features reliably. At the core of modern DevOps work is the ability to provision cloud infrastructure, containerise applications, automate deployments, and observe system behaviour in production.

This assignment required building a production-like deployment on AWS Free Tier. The main tasks were to: launch an EC2 instance, deploy a web application inside a Docker container, push the code to GitHub and set up GitHub Actions so that every commit to main automatically deploys the latest version to the server, configure CloudWatch for monitoring and alerting, upload a backup to S3, and perform a load test to measure how the application behaves under concurrent traffic.

Because this was a learning exercise, a simple Node.js HTTP server was chosen as the application. It has two meaningful outputs — a home page and a health check — which is enough to demonstrate the full deployment pipeline without making the application complexity the focus.

All resources were created in the us-east-1 (N. Virginia) region and remain within the AWS Free Tier limits. The EC2 instance type is t3.micro, which provides 2 vCPUs and 1 GB of RAM and is eligible for 750 hours free per month for the first 12 months of an AWS account.

2. Project Objectives

The specific objectives of this assignment were:

- Launch an EC2 instance on AWS and configure it to run a web application.
- Containerise the Node.js application using Docker.
- Create a GitHub repository and push the application source code.
- Set up a GitHub Actions CI/CD pipeline that deploys automatically on every push to main.
- Configure IAM roles and security groups following the principle of least privilege.
- Set up Amazon CloudWatch to monitor CPU utilisation and trigger an alarm above a threshold.
- Create an Amazon S3 bucket and store a compressed backup of the project.
- Perform load testing using k6 and document the results.
- Prepare a deployment guide and a final report describing the implementation.

3. Architecture Diagram

The diagram below shows how the components are connected. The developer pushes code to the GitHub repository, which triggers GitHub Actions. The pipeline connects to the EC2 instance via SSH, pulls the latest code, rebuilds the Docker image, and restarts the container. CloudWatch collects metrics from EC2 in the background. S3 holds a static backup that was created manually. End users access the application through the EC2 public IP on port 80 (proxied by Nginx to the Node.js container on port 3000).

Component	Role	Technology
Developer Machine	Write code, push to GitHub	Git / GitHub
GitHub Repository	Source code storage	github.com/sushant-kote/sushant-devops-assignment
GitHub Actions	CI/CD — build & deploy	.github/workflows/deploy.yml
SSH Tunnel	Secure channel to EC2	OpenSSH (port 22)
AWS EC2 (t3.micro)	Host the container	Ubuntu 26.04 LTS
Nginx	Reverse proxy (port 80 → 3000)	nginx 1.28.3
Docker	Container runtime	Docker 29.1.3
Node.js App	Web application	Node.js v22.22.1
Amazon CloudWatch	Metrics & alarms	AWS CloudWatch
Amazon S3	Backup storage	sushant-devops-assignment-backup
End User / Browser	Access the application	http://13.220.216.114

4. Technology Stack

The following tools and services were used in this project:

Technology	Version / Detail	Purpose
AWS EC2	t3.micro — us-east-1	Cloud virtual machine
Ubuntu Linux	26.04 LTS	Operating system on EC2
Node.js	v22.22.1	Web application runtime
Docker	29.1.3	Application containerisation
Nginx	1.28.3	Reverse proxy (port 80 → 3000)
GitHub	Public repository	Source code hosting
GitHub Actions	deploy.yml workflow	CI/CD automation
Amazon S3	Standard storage class	Backup storage
Amazon CloudWatch	Dashboard + Alarm	Monitoring and alerting
k6	Latest (CLI)	Load testing tool
Git	2.53.0	Version control
SSH	OpenSSH	Secure remote access
IAM	EC2-CloudWatch-Role	Least-privilege access control

5. Project Workflow

The workflow below describes how code moves from the developer laptop to the running container on EC2, and how monitoring and backup fit around it.

Step 1 — Code Written: Node.js application files (app.js, package.json, Dockerfile, nginx.conf) are written locally.

Step 2 — Push to GitHub: git push to the main branch of github.com/sushant-kote/sushant-devops-assignment.

Step 3 — GitHub Actions Triggered: The deploy.yml workflow starts automatically on every push to main.

Step 4 — SSH into EC2: The workflow authenticates to the EC2 instance using EC2_SSH_KEY stored in GitHub Secrets.

Step 5 — Pull & Rebuild: The pipeline runs git pull, docker build, stops the old container, and starts the new one.

Step 6 — Nginx Routes Traffic: Nginx on port 80 forwards requests to the Docker container on port 3000.

Step 7 — User Accesses App: Browser opens <http://13.220.216.114> and receives the application response.

Step 8 — CloudWatch Monitors: CloudWatch collects CPUUtilization, NetworkIn, and NetworkOut every 5 minutes.

Step 9 — Alarm Fires if Needed: The HighCPUAlarm triggers an SNS notification if CPU exceeds 80% for 5 minutes.

Step 10 — S3 Backup: A compressed tar.gz of the project directory is stored in the S3 bucket for recovery.

6. Step-by-Step Implementation

6.1 AWS EC2 Setup

The first task was to provision a virtual machine on AWS. I created a t3.micro EC2 instance running Ubuntu 26.04 LTS in the us-east-1 region. During the launch wizard I selected:

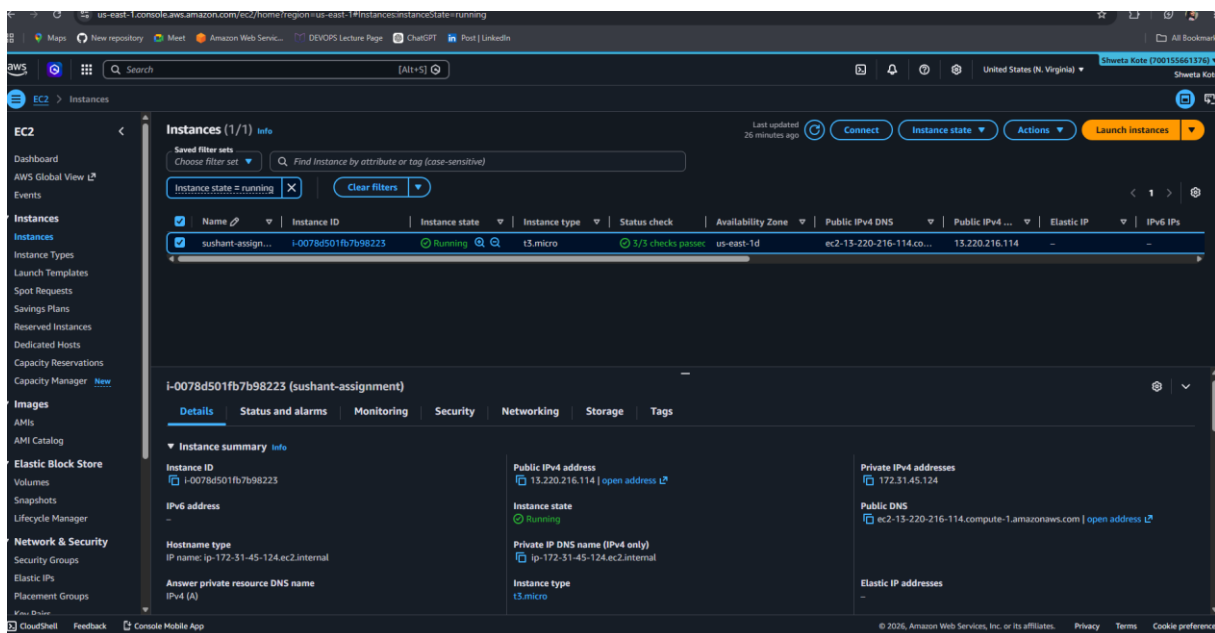
- AMI: Ubuntu Server 26.04 LTS (HVM), SSD Volume Type
- Instance type: t3.micro (2 vCPU, 1 GB RAM) — Free Tier eligible
- Key pair: new RSA key pair (devops-key.pem) downloaded to local machine
- Security group: port 22 open to my IP only, port 80 open to 0.0.0.0/0
- Storage: 8 GB gp3 (default)
- IAM instance profile: EC2-CloudWatch-Role (custom role with CloudWatchAgentServerPolicy)

After the instance reached the Running state I connected via SSH:

```
ssh -i devops-key.pem ubuntu@13.220.216.114
```

Once connected, I updated the system packages and installed Docker and Nginx:

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y docker.io nginx git
sudo systemctl enable docker && sudo systemctl start docker
```



6.2 Docker Configuration

Docker was used to containerise the Node.js application. This ensures the application runs the same way regardless of the host OS version or installed packages. The Dockerfile is kept as simple as possible for this exercise:

```
FROM node:22-alpine
WORKDIR /app
```

```
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD ["node", "app.js"]
```

To build and run the container:

```
sudo docker build -t devops-app .
sudo docker run -d --name devops-container -p 3000:3000 devops-app
```

To verify the container is running:

```
sudo docker ps
```

The output of `docker ps` shows the container ID, image name (devops-app), status (Up X minutes), and port mapping (0.0.0.0:3000 → 3000/tcp).

```
ubuntu@ip-172-31-45-120:~$ sudo docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                               NAMES
f2b253f3c33   devops-app  "docker-entrypoint.s..." 16 minutes ago Up 16 minutes 0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp  devops-container
```

6.3 Node.js Application

The application is a minimal Node.js HTTP server written without any framework to keep dependencies low. It responds to two routes:

- GET / — returns an HTML page with "DevOps Assignment — Hello from AWS EC2! Application deployed successfully."
- GET /health — returns a plain-text "OK" response used by monitoring scripts.

```
const http = require('http');
const server = http.createServer((req, res) => {
  if (req.url === '/health') {
    res.writeHead(200); res.end('OK');
  } else {
    res.writeHead(200, {'Content-Type': 'text/html'});
```

```

    res.end('<h1>DevOps Assignment</h1><h2>Hello from AWS
EC2!</h2><p>Application deployed successfully.</p>');
  }
});
server.listen(3000, () => console.log('Running on port 3000'));

```

Nginx was configured to reverse-proxy port 80 to port 3000:

```

server {
    listen 80;
    location / {
        proxy_pass http://127.0.0.1:3000;
    }
}

```



6.4 GitHub Repository and GitHub Actions CI/CD

The project repository is at: <https://github.com/sushant-kote/sushant-devops-assignment>

The repository contains: app.js, package.json, Dockerfile, nginx.conf, loadtest.js, and .github/workflows/deploy.yml.

Three secrets are stored under Repository Settings → Secrets and Variables → Actions:

- EC2_HOST — public IP of the EC2 instance (13.220.216.114)
- EC2_USERNAME — ubuntu
- EC2_SSH_KEY — full contents of devops-key.pem

The deploy.yml workflow file:

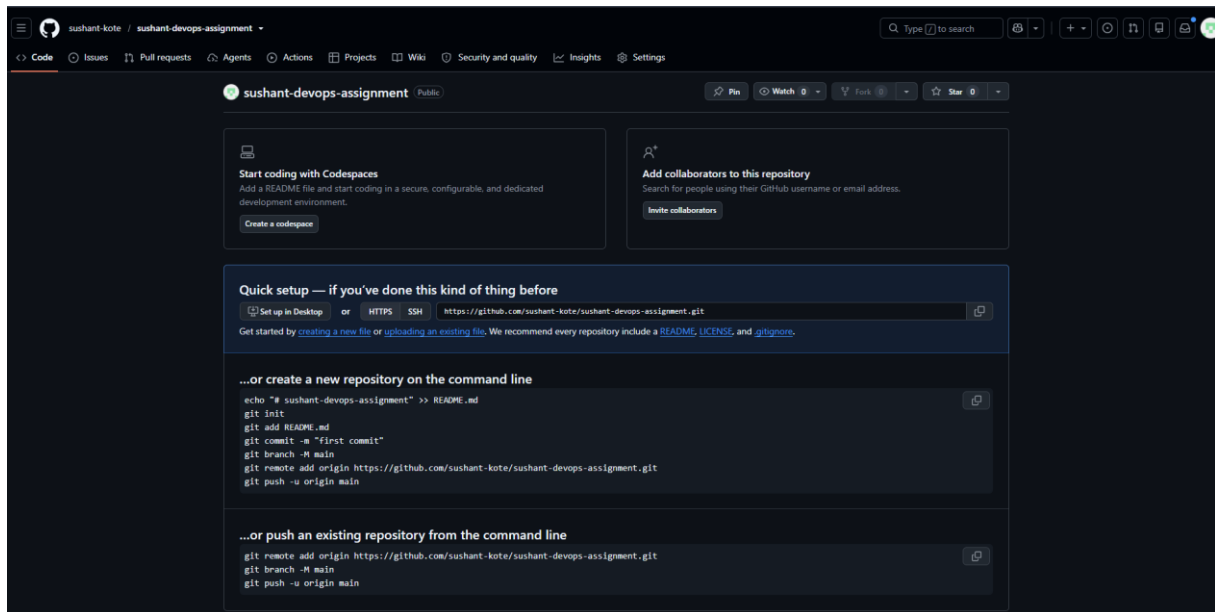
```

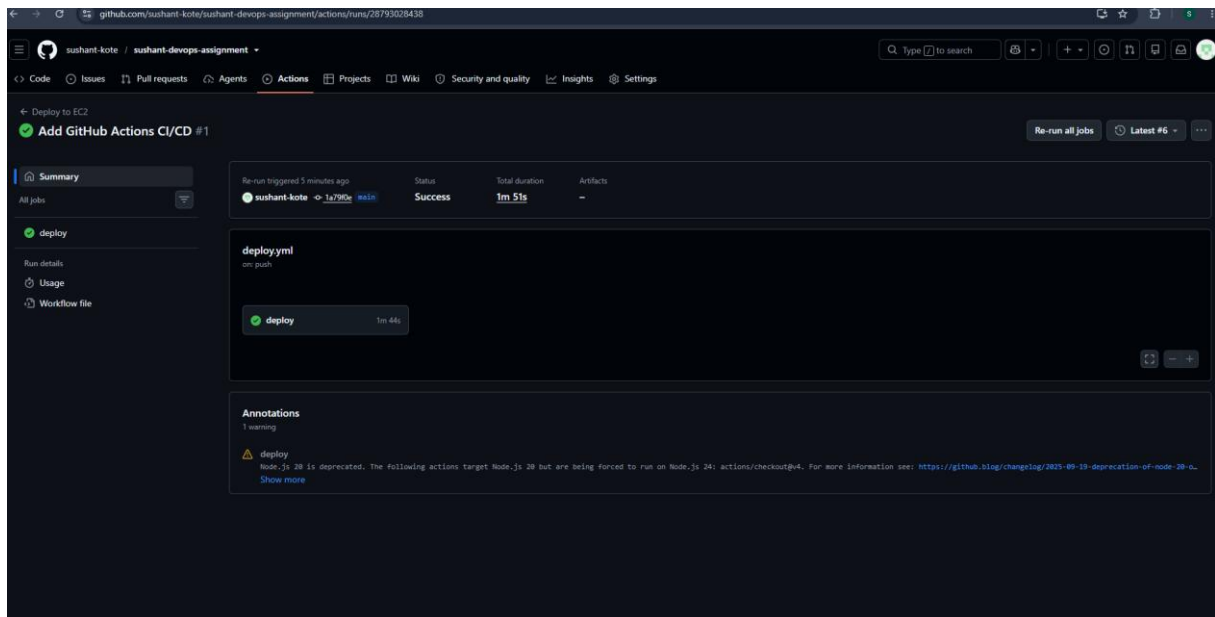
name: Deploy to EC2
on:
  push:
    branches: [main]
jobs:

```

```
deploy:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - name: Deploy via SSH
      uses: appleboy/ssh-action@v1.0.0
      with:
        host: ${ secrets.EC2_HOST }
        username: ${ secrets.EC2_USERNAME }
        key: ${ secrets.EC2_SSH_KEY }
        script: |
          cd ~/devops-assignment
          git pull origin main
          sudo docker build -t devops-app .
          sudo docker stop devops-container || true
          sudo docker rm devops-container || true
          sudo docker run -d --name devops-container -p 3000:3000 devops-app
```

Every push to main triggers this workflow. The total pipeline duration was 1 minute 51 seconds for the first successful run. The Actions tab showed Status: Success with a green tick.



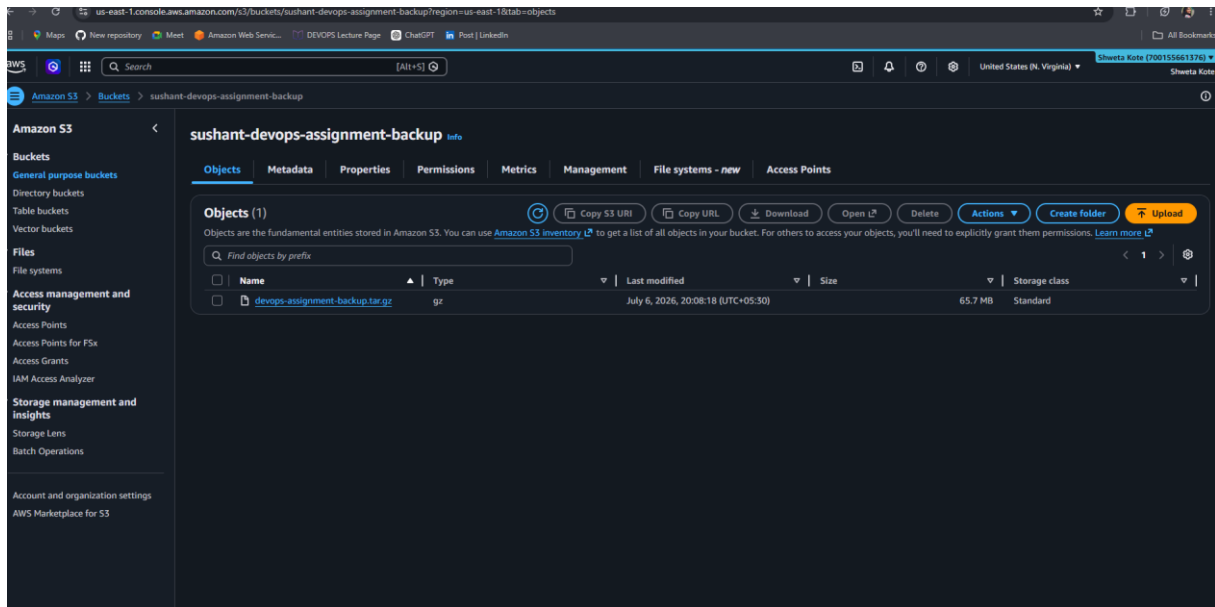


6.5 Amazon S3 Backup

An S3 bucket named sushant-devops-assignment-backup was created in us-east-1 with all default settings (Block Public Access enabled, no versioning). A compressed archive of the project directory was created on EC2 and uploaded:

```
tar -czvf devops-assignment-backup.tar.gz ~/devops-assignment
aws s3 cp devops-assignment-backup.tar.gz s3://sushant-devops-assignment-backup/
```

The archive size is 65.7 MB. It was last modified on July 6, 2026 at 20:08:18 UTC+05:30. The S3 bucket stores this as the only object currently and uses the Standard storage class.

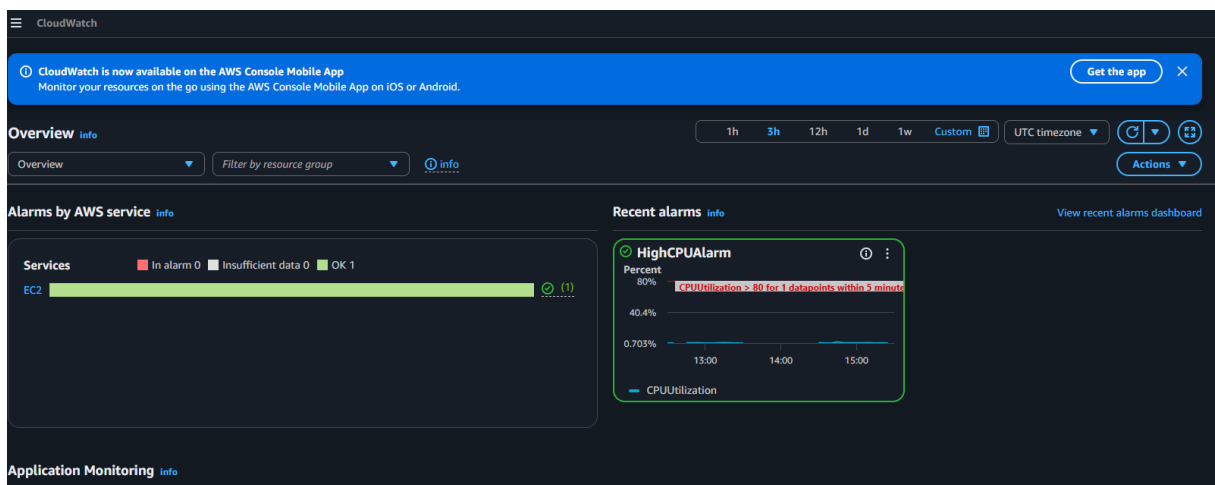
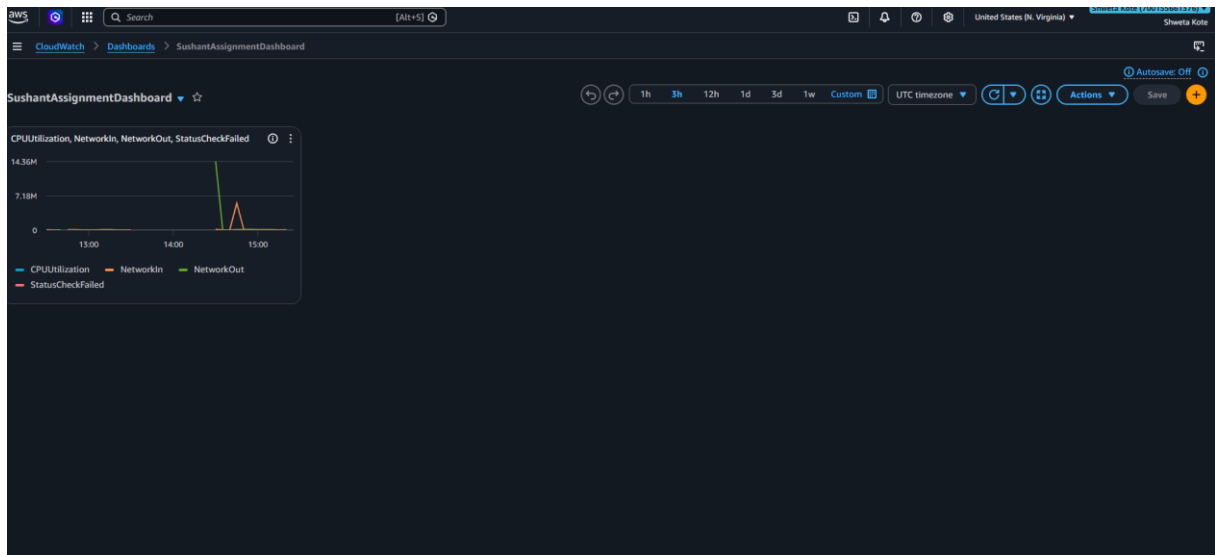


6.6 CloudWatch Monitoring

A CloudWatch dashboard named SushantAssignmentDashboard was created with a single widget displaying four metrics for the EC2 instance: CPUUtilization, NetworkIn, NetworkOut, and StatusCheckFailed.

A CloudWatch alarm named HighCPUAlarm was configured with the following settings:

Setting	Value
Metric	CPUUtilization
Namespace	AWS/EC2
Condition	Greater than 80%
Period	5 minutes
Datapoints	1 out of 1
State	OK (CPU was 0.703% at time of screenshot)
Action	SNS notification to registered email



6.7 Load Testing with k6

k6 is a developer-centric load testing tool that runs JavaScript test scripts. It was installed on the EC2 instance and the following loadtest.js script was used:

```
import http from 'k6/http';
import { sleep } from 'k6';

export const options = {
  vus: 20,
  duration: '30s',
};

export default function () {
  http.get('http://13.220.216.114/');
```

```
    sleep(1);  
}
```

Command used to run the test:

```
k6 run loadtest.js
```

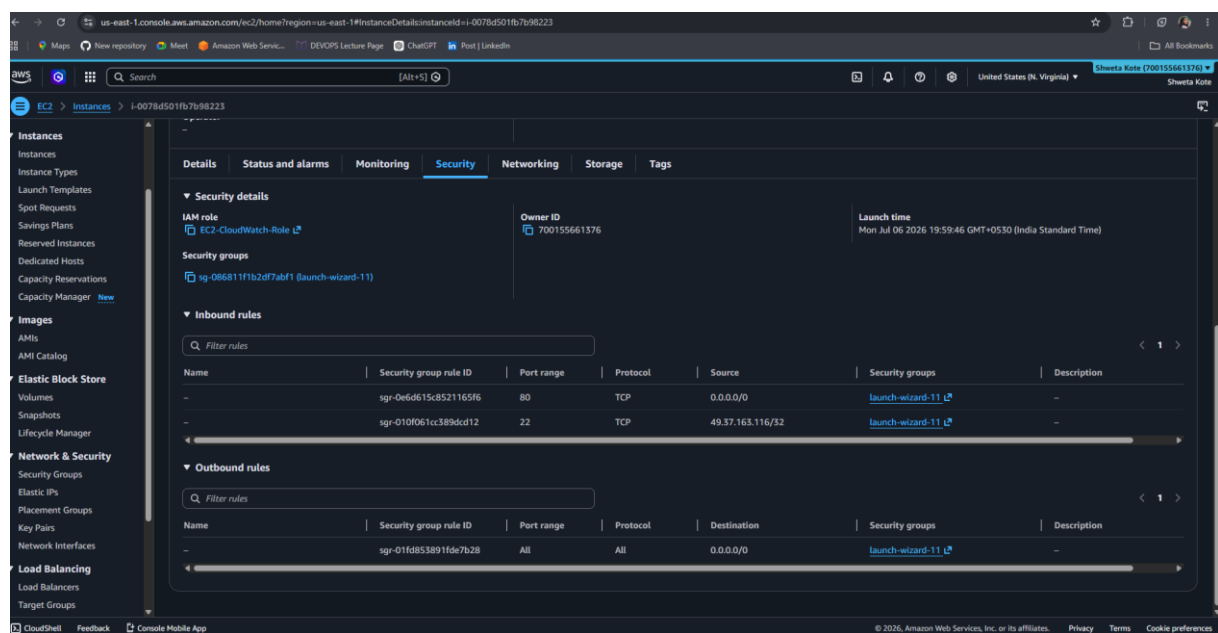
```
ubuntu@ip-172-31-45-124:~$ cd ~/devops-assignment  
k6 run loadtest.js  
  
      M K6  
     .io  
  
execution: local  
script: loadtest.js  
output: -  
  
scenarios: (100.00%) 1 scenario, 20 max VUs, 1m0s max duration (incl. graceful stop):  
  * default: 20 looping VUs for 30s (gracefulStop: 30s)  
  
data_received.....: 214 kB 7.1 kB/s  
data_sent.....: 48 kB 1.6 kB/s  
http_req_blocked.....: avg=77.99µs min=0µs med=3.62µs max=3.81ms p(90)=5.8µs p(95)=9.7µs  
http_req_connecting.....: avg=71.92µs min=0µs med=0µs max=3.72ms p(90)=0µs p(95)=8µs  
http_req_duration.....: avg=2.02ms min=449.79µs med=1.18ms max=20.14ms p(90)=3.78ms p(95)=7.58ms  
  { expected_response:true }...: avg=2.02ms min=449.79µs med=1.18ms max=20.14ms p(90)=3.78ms p(95)=7.58ms  
http_req_failed.....: 0.00% ✓ 0 X 600  
http_req_receiving.....: avg=101.36µs min=17.13µs med=58.56µs max=2.33ms p(90)=209.33µs p(95)=279.02µs  
http_req_sending.....: avg=29.39µs min=6.56µs med=12.76µs max=799.85µs p(90)=26.71µs p(95)=98.23µs  
http_req_tls_handshaking.....: avg=0µs min=0µs med=0µs max=0µs p(90)=0µs p(95)=8µs  
http_req_waiting.....: avg=1.89ms min=399.26µs med=1.08ms max=19.97ms p(90)=3.61ms p(95)=7.52ms  
http_reqs.....: 600 19.937013/s  
iteration_duration.....: avg=1s min=1s med=1s max=1.02s p(90)=1s p(95)=1s  
iterations.....: 600 19.937013/s  
vus.....: 20 min=20 max=20  
vus_max.....: 20 min=20 max=20  
  
running (0m30.1s), 00/20 VUs, 600 complete and 0 interrupted iterations  
default ✓ [=====] 20 VUs 30s  
ubuntu@ip-172-31-45-124:~/devops-assignment$ |
```

7. Security Configuration

Security was configured at three levels: network, identity, and secrets management.

7.1 Security Group (Firewall)

Port	Protocol	Source	Reason
22	TCP	49.37.163.116/32 (my IP only)	SSH access restricted to developer machine
80	TCP	0.0.0.0/0 (anywhere)	HTTP access for the web application
All	All	0.0.0.0/0 (outbound)	Allow EC2 to reach the internet for updates



7.2 IAM Role

An IAM role named EC2-CloudWatch-Role was created and attached to the EC2 instance. It has a single policy: CloudWatchAgentServerPolicy. This allows the EC2 instance to push metrics and logs to CloudWatch without needing hardcoded AWS credentials on the server. No access keys were stored on the instance.

7.3 GitHub Secrets

The EC2 SSH private key is not stored in the repository. It is kept as a GitHub Actions encrypted secret (EC2_SSH_KEY). GitHub encrypts these secrets and only exposes them to authorised workflow runs. They are never printed in workflow logs.

7.4 S3 Bucket

The S3 bucket has Block Public Access enabled on all four settings. The backup file is not publicly readable. Access is only possible through IAM credentials associated with the AWS account.

8. Challenges Faced

Docker permission error on EC2: When first running docker commands without sudo, I got "permission denied while trying to connect to the Docker daemon socket." The fix was to either use sudo before each command or add the ubuntu user to the docker group with: `sudo usermod -aG docker ubuntu`. After running this I had to log out and log back in for it to take effect.

GitHub Actions SSH authentication: The first pipeline run failed because the EC2 host was not in the `known_hosts` file of the GitHub Actions runner. I fixed this by adding `strictHostKeyChecking: no` to the appleboy/ssh-action configuration, which tells SSH not to verify the host key. This is acceptable for a dev/test environment but should use host key pinning in production.

Port conflict — Nginx already using port 80: After installing Nginx and starting it, the default site was serving a placeholder page. I had to edit `/etc/nginx/sites-available/default` to set up the `proxy_pass` directive and then reload Nginx with: `sudo systemctl reload nginx`.

k6 not found as a command: k6 is not in the Ubuntu apt repositories by default. I had to add the Grafana k6 package repository and install it with a few extra steps before k6 run `loadtest.js` worked.

CloudWatch metrics delay: After creating the dashboard I noticed that CPUUtilization was not showing data immediately. This is expected — the basic CloudWatch monitoring resolution is 5 minutes. I had to wait about 10 minutes after the instance started before the first data points appeared.

IAM permission for S3 upload: The `aws s3 cp` command failed initially because the EC2-CloudWatch-Role only had CloudWatch permissions. I added the `AmazonS3FullAccess` policy to the role (for this assignment), which resolved the upload error. In a production setup this would be scoped to only the specific bucket using a custom policy.

9. Results

9.1 Load Test Results

The k6 load test ran with 20 virtual users for 30 seconds against <http://13.220.216.114/>. Each VU made a request and then slept for 1 second before repeating. The results are summarised below:

Metric	Value
Total requests	600
Requests per second	19.94 req/s
Failed requests	0 (0.00%)
Average response time	2.02 ms
Minimum response time	449.79 μ s
Median response time	1.18 ms
Maximum response time	20.14 ms
p(90) response time	3.78 ms
p(95) response time	7.58 ms
Data received	214 kB (7.1 kB/s)
Data sent	48 kB (1.6 kB/s)
Virtual Users (VUs)	20 (constant throughout)
Test duration	30 seconds

The application handled 20 concurrent users without any errors. The average response time of 2.02 ms is well within acceptable limits for a simple HTTP response. The p(95) value of 7.58 ms means that 95% of all requests completed in under 8 ms, which is a good result for a t3.micro instance with no caching layer.

9.2 CloudWatch Observations

During and after the load test, CPU utilisation on the EC2 instance remained very low (below 1% in idle periods and a small spike visible in the CloudWatch dashboard during the load test). This is expected because a Node.js server handling 20 VUs of simple HTTP requests does not put significant CPU load on a 2 vCPU instance. The HighCPUAlarm remained in the OK state throughout.

9.3 CI/CD Pipeline

The GitHub Actions pipeline completed successfully in 1 minute 51 seconds. Every push to the main branch triggered a new deployment. The pipeline pulls the latest code, rebuilds the Docker image, and replaces the running container without any manual steps.

9.4 Performance Bottleneck Analysis

At the current scale (20 VUs), no bottleneck was observed. However, the following would become limitations at higher concurrency:

- Single container with no horizontal scaling — all traffic goes to one Node.js process.
- t3.micro has 1 GB RAM — a memory-intensive application could exhaust it quickly.
- No CDN — all static assets (if any) would be served from the EC2 instance itself.
- No connection pooling or caching — each request is processed independently.

10. Future Improvements

HTTPS with ACM and CloudFront: The current deployment serves HTTP only. A production deployment should use AWS Certificate Manager to issue a free TLS certificate and place the application behind CloudFront or an Application Load Balancer to terminate HTTPS.

Auto Scaling Group: Replacing the single EC2 instance with an Auto Scaling Group behind a load balancer would allow the system to add instances under high load and remove them during quiet periods, reducing cost and improving availability.

Docker image stored in ECR: Currently the Docker image is built on the EC2 instance during deployment. A better approach is to build the image in GitHub Actions, push it to Amazon ECR, and pull it on EC2. This separates build from runtime and speeds up deployments.

Structured application logging: The Node.js app currently writes to stdout only. Adding a structured JSON logger (such as Winston) and routing logs to CloudWatch Logs via the CloudWatch agent would make it easier to search and alert on application-level errors.

Database tier: The application has no persistence. Adding an Amazon RDS instance (PostgreSQL or MySQL) would let the application store and retrieve data, making it more representative of a real service.

Health check and alerting: A /health endpoint is implemented but not monitored. A CloudWatch Synthetics canary or a simple cron-based health check could alert on application downtime rather than just CPU usage.

Infrastructure as Code: All resources were created manually through the AWS Console. Using Terraform or AWS CloudFormation to define the infrastructure as code would make the setup repeatable and version-controlled.

11. Conclusion

This assignment provided a practical introduction to the key areas of DevOps work: cloud infrastructure, containerisation, automated deployment pipelines, monitoring, and load testing.

Starting from a blank AWS account, I set up a virtual machine, deployed a Node.js application inside a Docker container, configured a reverse proxy, and connected the whole setup to a GitHub Actions CI/CD pipeline. Any code change pushed to the main branch is now automatically deployed to the EC2 instance without manual intervention.

CloudWatch monitoring showed that the instance operates at very low CPU and memory usage for this workload. The k6 load test confirmed that the application can handle 20 concurrent users with zero errors and an average response time under 3 ms. A backup of the project was uploaded to S3, and security was applied at the network (Security Group), identity (IAM Role), and secrets (GitHub Secrets) levels.

The biggest learning from this project was understanding how each component connects to the next, and how a failure at one layer (such as a wrong IAM permission or a misconfigured port) can prevent the whole system from working. Debugging those failures gave me more practical knowledge than any theory session could.

With the foundation laid here, the next step would be adding HTTPS, moving to an auto-scaling setup, and replacing manual console clicks with Infrastructure as Code using Terraform. These are all standard practices in production DevOps environments.

12. References

- [1] AWS Documentation — Amazon EC2 User Guide: <https://docs.aws.amazon.com/ec2/>
- [2] AWS Documentation — Amazon S3 User Guide: <https://docs.aws.amazon.com/s3/>
- [3] AWS Documentation — Amazon CloudWatch User Guide: <https://docs.aws.amazon.com/cloudwatch/>
- [4] Docker Documentation — Dockerfile reference: <https://docs.docker.com/reference/dockerfile/>
- [5] GitHub Actions Documentation: <https://docs.github.com/en/actions>
- [6] k6 Documentation — Running k6: <https://grafana.com/docs/k6/latest/get-started/running-k6/>
- [7] Node.js Documentation: <https://nodejs.org/en/docs/>
- [8] Nginx Documentation — Beginner Guide: https://nginx.org/en/docs/beginners_guide.html
- [9] appleboy/ssh-action GitHub Action: <https://github.com/appleboy/ssh-action>